

Mid

C as a target language for a compiler == source-to-source translator == transpiler

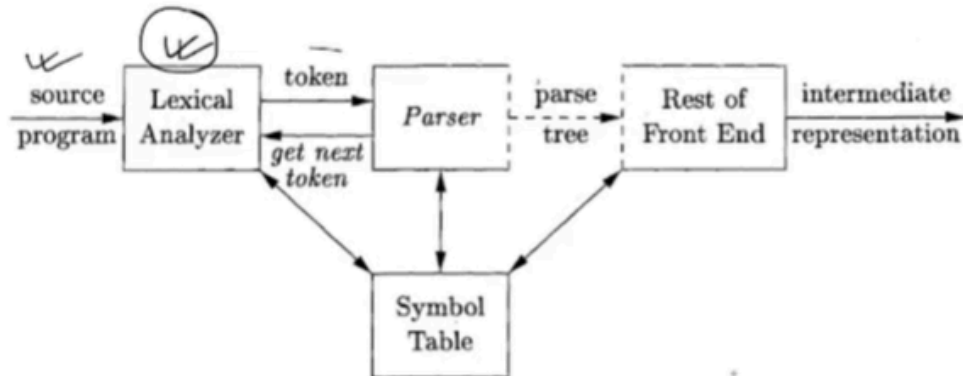


Figure 4.1: Position of parser in compiler model

Final

4. Consider the following **SDT**. Draw the **Annotated Parse Tree** and **Evaluate** it for the String: *while* ($x < y$) $z = u + v - x$; $y = z * u$;

Note: The newlabel() and newtemp() function details have been given to you

(5 points for parse tree + 5 points for proper annotation of labels and temp variables for expressions in the parse tree + 5 points for final three address code)

SDT:

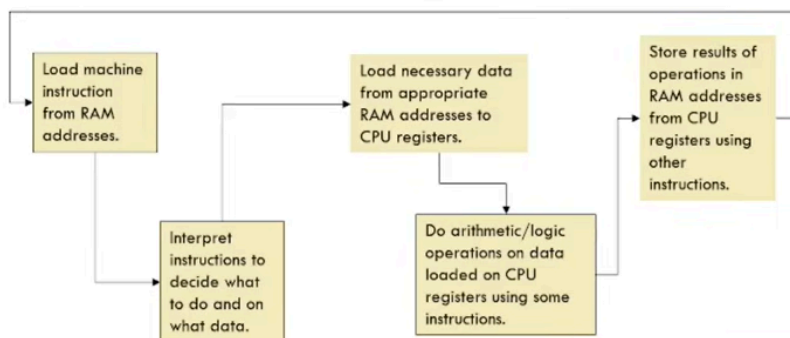
```
int newlabel() {  
    return "L" || p;  
}  
int newtemp() {  
    return 't' || p;  
}
```

```
P -> {p = 0; S.next = newlabel();} S {P.code = S.code || label(S.next)}  
S -> {p++;} Assign {S.code = Assign.code;}  
Assign -> id = E; {Assign.code = E.code || gen(id.lexeme = E.addr);}  
S -> if ( {p++; B.true = newlabel(); B.false = S1.next = S.next;}  
    B ) {p++;  
    S1 {S.code = B.code || label(B.true) || S1.code;}  
S -> if ( {p++; B.true = newlabel() || "_T"; B.false = newlabel() || "_F";}  
    B ) {p++; S1.next = S.next} S1 else {p++; S2.next = S.next}  
    S2 {S.code = B.code || label(B.true) || S1.code || gen('goto' S.next) ||  
        label(B.false) || S2.code;}  
S -> while ( {p++; begin = newlabel(); B.true = newlabel() || "_T"; B.false = S.next;}  
    B ) {p++; S1.next = begin;}  
    S1 {S.code = label(begin) || B.code || label(B.true) || S1.code || gen('goto'  
        begin);}  
S -> {p++; S1.next = newlabel();}  
    S1 {p++; S2.next = S.next;}  
    S2 {S.code = S1.code || label(S1.next) || S2.code;}  
B -> {p++;}  
    E1 rel {p++;}  
    E2 {B.code = E1.code || E2.code  
        || gen('if' E1.addr rel.op E2.addr 'goto' B.true); || gen('goto' B.false)}  
E -> {p++;}  
    E1 arith {p++;}  
    E2 {E.addr = newtemp(); E.code = E1.code || E2.code || gen(E.addr = E1.addr  
        arith.op E2.addr);}  
E -> id {E.addr = id.lexeme; E.code = "";}  
E -> num {E.addr = num.value; E.code = "";}
```

```

L2:    if x < y goto L2_T
      goto L1
L2_T:  t7 = u + v
      t7 = t7 - x
      z = t7
      goto L2
L1:    t10 = z * u
      y = t10
L0:

```



Memory Access: Load				
Op	Rs1	vs	Offset	
4	3	3	6	
Memory Access: Store				
Op	Rs1	Rs2	Offset	
4	3	3	6	
Data Processing:				
Op	Rs1	Rs2	vs	Offset
4	3	3	3	3
Branch:				
Op	Rs1	Rs2	Offset	
4	3	3	6	
Jump:				
Op	Offset			
4	12			

OP	
0000	Load Word
0001	Store Word
0002	Add
0003	Subtract
0004	Insert
0005	Logical Shift Left
0006	Logical Shift Right
0007	Bitwise AND
0008	Bitwise OR
0009	Set on less than
0010	Hamming Distance
0011	Branch on Equal
0012	Branch on NOT Equal
0013	Jump

Static address allocation of variables does not work for variables that are local to functions.

This fact can be understood from nested function calls.

However, recursive functions illustrates this concepts very well.

foo(5)
b = 25
c = 10
return b + c + foo(4)
25 + 10 + 8
b = 16
c = 8

```
void main() {
    int x = 5;
    int y = foo(x);
    y = y / x;
}

int foo(int a) {
    int b = a * a;
    int c = a + a;
    if (c < b / 2) {
        return b + c + foo(a - 1);
    } else {
        return b - c;
    }
}
```

Consider this sample recursive code

Assumed static addresses for local variable b and c of function foo

b = 25 then 16
c = 10 then 8

Main
x = 5
foo(4)
b = 25
c = 10
foo(9)
b = 16
c = 8

The problem if the call from foo(5) to foo(4) will override b and c from 25 and 10 to 16 and 8 if the address is static. So we cannot go back to foo(5) and return 25 + 10 + 8 to main.

BY DR. MUHAMMAD HUR YAHRAONA 12

THE IDEA OF FUNCTION CALL STACK

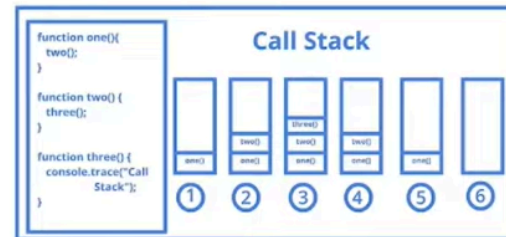
The lesson we learn from the previous example is that each time a function call happens, we must find new space in the logical address space of the process to allocate its local variables.

This gives rise to the concepts of stacks.

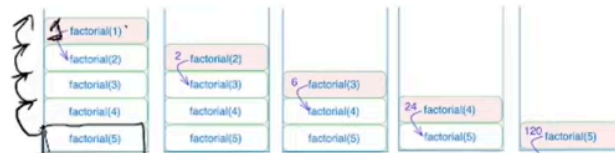
All necessary information related to function calls: including function parameters, location for return values, function's local variables, and instruction of the caller to return to when a function completes executions is stored in a stack. This information is called **function activation record**.

As function calls other functions, stack grows.

As the last called function returns, its space in the stack is freed and the code returns to the previous function using information stored in the stack.



Function activation records for nested function calls.



Function activation records for recursive function calls.

Absolute Syntax Tree ?

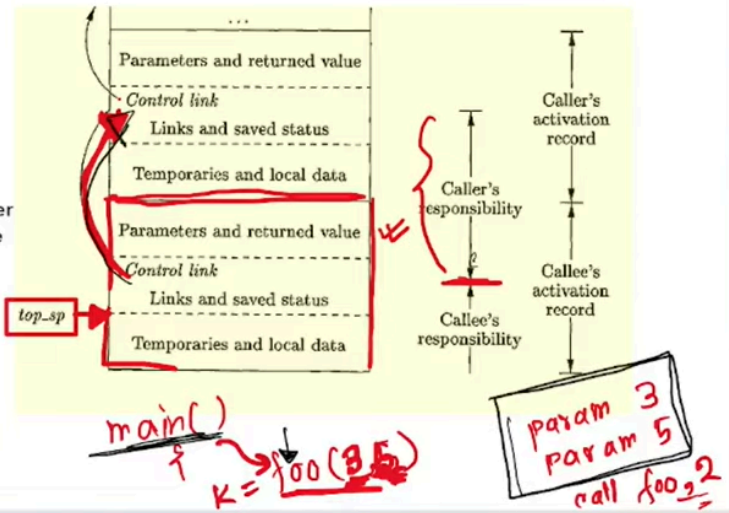
Annotated parse Tree er Compact version

CALLER AND CALLEE RESPONSIBILITY BREAKDOWN

When compiler generates code for functions, it has to decide how to distribute the task of populating different parts of a function activation records between the caller function and the callee function.

Having the code in the callee allows the compiler to generate the code for a part only once to be used by many caller function's code.

However, things that the callee should not know beforehand must be populated by the caller.



Q/A

```
=====
Helper Functions (Pseudocode)
=====
```

Note: Alignment affects offset placement, not the logical order of declarations. Inside records, both **each field** is aligned before placement, and the **total record size** is padded to *record.align*

```
func alignTo(off, a) {
    return ((off + a - 1) / a) * a;
}
```